Calendar Reminder

A Mini Project Presentation presented
to the College of Information Technology Education
PHINMA – University of Iloilo
Iloilo City

Submitted in partial of the requirements for the degree of
Bachelor of Science in Information Technology
(ITE 260 Computer Programming 1)

Balajadia, Rain
Celino, Edison E.
Guevarra, Bimbo
Isiderio, Alexis
Pesito, Chenalyn
Members

October, 2026

Introduction

Description

This chapter presents the objectives of the Calendar Reminder program. It explains important concept used like list, start, and the repetition of command. It also gives short explanation about the how the program were made. The purpose of this chapter is for the users to easily understand the importance and use of the program.

In this part, it explains the background of the study that were made. The Calendar Reminder is a simple GUI application that's built using Python. It aims to give easy and organized way of recording important date and time of important tasks. This is not a complex system, but a valuable system that can be used by anyone to keep them on track of their important tasks.

Objectives of the Study

General Objective

To develop a Calendar Reminder

Specific Objectives

In particular, this study aims to achieve the following:

1. Create a program that allow users to input information like date, time, and reminder notes.
2. Can show a list of all listed reminders in order and clear way.
3. Can delete reminders that's not needed by the users to maintain list order.

Scope and Limitations

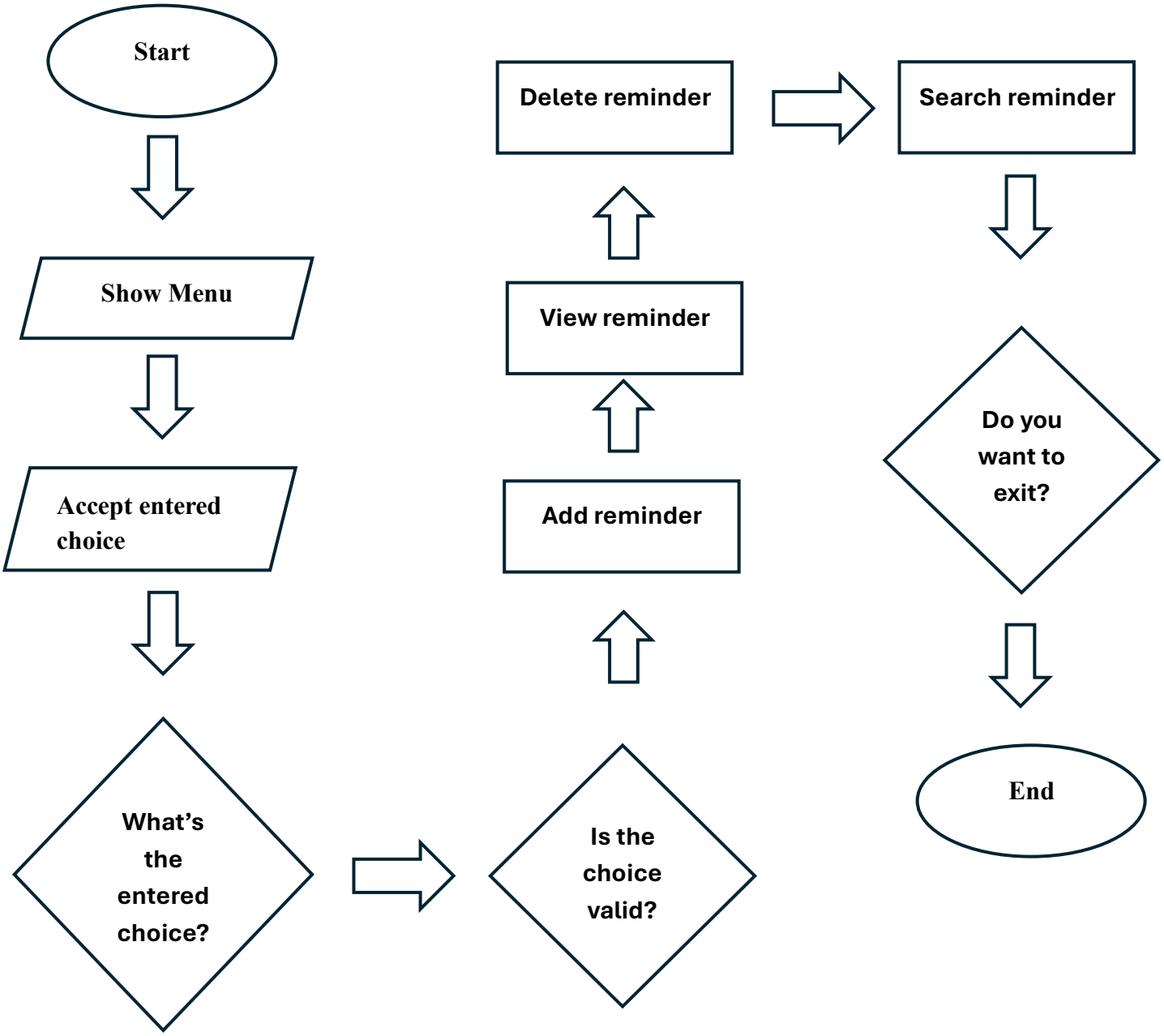
The scope of this study only focuses on adding, viewing, and deleting reminders. Each reminders consist of three parts: date, time, and short note. The program only works using text or console. The data will remain while the program is running—if it's closed, the input data will vanish because the program doesn't have permanent data (something, forgot the term), it also does not send alarm or notification.

Definition of Terms

• **Python:** Language in programming that's used for creating this project. It's known as easy to read and understand.

• **List:** Type of storage in Python that's used to combine all reminders in one place.
• **Function:** A block of command that follows a specific task — it's used to divide the program in an easy way like adding and viewing reminders.
• **Loop:** Structure that repeatedly shows the menu up until the user decided to exit the program.
• **Variable:** A container where the information is stored temporarily like date, time, and note of reminder.

Flow of the Data



Key Features

The program can create data by accepting information from the user. Here's the key feature of the program:

• Can add new reminder with date, time, and note.
• Can search and show all listed reminder.
• Can delete reminders that's not needed anymore.
• Can search reminders

Source Code

```
import tkinter as tk
from tkinter import messagebox, simpledialog

# Calendar Reminder
reminders = []


# Add Reminder
def add_reminder():
    date = date_entry.get()
    time = time_entry.get()
    note = note_entry.get()

    if date == "" or time == "" or note == "":
        messagebox.showwarning(
            "Warning", "Please fill in all fields."
        )
        return

    reminder = {
        "date": date,
        "time": time,
        "note": note
    }

    reminders.append(reminder)
```

```
    messagebox.showinfo(
        "Success", "Reminder saved successfully!"
    )

    date_entry.delete(0, tk.END)
    time_entry.delete(0, tk.END)
    note_entry.delete(0, tk.END)

    view_reminder()


# View Reminders
def view_reminder():
    reminder_list.delete(0, tk.END)

    if len(reminders) == 0:
        reminder_list.insert(tk.END, "No reminders found.")
    else:
        for i, reminder in enumerate(reminders, start=1):
            display = (
                f"{i}. {reminder['date']} | "
                f"{reminder['time']} | "
                f"{reminder['note']}"
            )
            reminder_list.insert(tk.END, display)


# Search Reminder
def search_reminder():
    keyword = simpledialog.askstring(
        "Search Reminder",
        "Enter a keyword to search:"
    )

    if keyword is None:
        return
```

```
    keyword = keyword.lower()
    results = []

    for i, reminder in enumerate(reminders, start=1):
        if keyword in reminder["note"].lower():
            results.append(
                f"{i}. Date: {reminder['date']}\n"
                f"   Time: {reminder['time']}\n"
                f"   Note: {reminder['note']}"
            )

    if results:
        messagebox.showinfo(
            "Search Results", "\n\n".join(results)
        )
    else:
        messagebox.showinfo(
            "Search Results", "No matching reminder found."
        )


# Delete Reminder
def delete_reminder():
    if len(reminders) == 0:
        messagebox.showwarning(
            "Warning", "There are no reminders to delete."
        )
        return

    number = simpledialog.askinteger(
        "Delete Reminder",
        "Enter the reminder number to delete:",
        minvalue=1,
        maxvalue=len(reminders)
    )

    if number is None:
```

```
        return

    reminders.pop(number - 1)

    messagebox.showinfo(
        "Success", "Reminder deleted successfully!"
    )

    view_reminder()


# Exit Program
def exit_program():
    answer = messagebox.askyesno(
        "Exit", "Are you sure you want to exit?"
    )

    if answer:
        window.destroy()


# Main Window
window = tk.Tk()
window.title("Calendar Reminder")
window.geometry("600x600")
window.resizable(False, False)

# Title
title_label = tk.Label(
    window,
    text="CALENDAR REMINDER",
    font=("Arial", 20, "bold")
)
title_label.pack(pady=15)

# Input Frame
input_frame = tk.Frame(window)
```

```
input_frame.pack(pady=5)

# Date
tk.Label(
    input_frame,
    text="Date (MM/DD/YYYY):",
    font=("Arial", 11)
).grid(row=0, column=0, padx=10, pady=5, sticky="w")

date_entry = tk.Entry(input_frame, width=30)
date_entry.grid(row=0, column=1, padx=10, pady=5)

# Time
tk.Label(
    input_frame,
    text="Time (HH:MM AM/PM):",
    font=("Arial", 11)
).grid(row=1, column=0, padx=10, pady=5, sticky="w")

time_entry = tk.Entry(input_frame, width=30)
time_entry.grid(row=1, column=1, padx=10, pady=5)

# Note
tk.Label(
    input_frame,
    text="Reminder:",
    font=("Arial", 11)
).grid(row=2, column=0, padx=10, pady=5, sticky="w")

note_entry = tk.Entry(input_frame, width=30)
note_entry.grid(row=2, column=1, padx=10, pady=5)

# Buttons Frame
button_frame = tk.Frame(window)
button_frame.pack(pady=15)

tk.Button(
```

```
    button_frame,
    text="Add Reminder",
    width=15,
    command=add_reminder
).grid(row=0, column=0, padx=5, pady=5)

tk.Button(
    button_frame,
    text="View Reminders",
    width=15,
    command=view_reminder
).grid(row=0, column=1, padx=5, pady=5)

tk.Button(
    button_frame,
    text="Search Reminder",
    width=15,
    command=search_reminder
).grid(row=1, column=0, padx=5, pady=5)

tk.Button(
    button_frame,
    text="Delete Reminder",
    width=15,
    command=delete_reminder
).grid(row=1, column=1, padx=5, pady=5)

# Reminder List
tk.Label(
    window,
    text="YOUR REMINDERS",
    font=("Arial", 13, "bold")
).pack(pady=5)

reminder_list = tk.Listbox(
    window,
    width=75,
```

```
    height=12
)
reminder_list.pack(padx=10, pady=5)

# Exit Button
tk.Button(
    window,
    text="Exit",
    width=15,
    command=exit_program
).pack(pady=10)

# Run Program
view_reminder()
window.mainloop()
```